```
import numpy as np

# ==========================================
# 1. OBJECTIVE FUNCTION (Supply Chain Costs)
# ==========================================
def inventory_objective_function(order_quantities, demand_forecast, holding_cost_per_unit, setup_cost_per_order, stockout_penalty_
    """
    Calculates the total supply chain cost given an order replenishment vector.
    Seeks to minimize: Ordering Cost + Holding Cost + Stockout Penalty Cost.
    """
    total_ordering_cost = 0
    total_holding_cost = 0
    total_stockout_cost = 0
    current_inventory = 0

    for t in range(len(demand_forecast)):
        # Trigger fixed setup costs if an order is placed (thresholded at > 0.5 units)
        if order_quantities[t] > 0.5:
            total_ordering_cost += setup_cost_per_order
            current_inventory += order_quantities[t]

        demand = demand_forecast[t]

        if current_inventory >= demand:
            current_inventory -= demand
            total_holding_cost += current_inventory * holding_cost_per_unit
        else:
            shortage = demand - current_inventory
            total_stockout_cost += shortage * stockout_penalty_per_unit
            current_inventory = 0

    return total_ordering_cost + total_holding_cost + total_stockout_cost


# ==========================================
# 2. PARTICLE DEFINITION
# ==========================================
class Particle:
    def __init__(self, dimensions, min_bound, max_bound):
        # Position vector maps to order quantity per period
        self.position = np.random.uniform(min_bound, max_bound, dimensions)
        # Control initial velocity scale relative to search space boundaries
        self.velocity = np.random.uniform(-abs(max_bound-min_bound)/10, abs(max_bound-min_bound)/10, dimensions)
```

```
        self.velocity = np.random.uniform(-abs(max_bound-min_bound)/10, abs(max_bound-min_bound)/10, dimensions)
        self.best_position = np.copy(self.position)
        self.best_score = float('inf')
        self.score = float('inf')

    def update_position(self, min_bound, max_bound):
        self.position += self.velocity
        # Clamp positions within allowed warehouse capacity constraints
        self.position = np.clip(self.position, min_bound, max_bound)


# ==========================================
# 3. PSO OPTIMIZATION RUNTIME
# ==========================================
def pso_stock_optimization(demand_forecast, holding_cost, setup_cost, penalty_cost, max_capacity,
                           num_particles, max_iterations, w, c1, c2):

    dimensions = len(demand_forecast)
    min_bound = 0.0
    max_bound = float(max_capacity)

    # Instantiate particle populations
    swarm = [Particle(dimensions, min_bound, max_bound) for _ in range(num_particles)]
    global_best_position = np.zeros(dimensions)
    global_best_score = float('inf')

    # Meta-heuristic convergence iterations
    for iteration in range(max_iterations):
        for particle in swarm:
            # Evaluate performance against the Objective Function
            particle.score = inventory_objective_function(
                particle.position, demand_forecast, holding_cost, setup_cost, penalty_cost
            )

            # Local personal best validation
            if particle.score < particle.best_score:
                particle.best_score = particle.score
                particle.best_position = np.copy(particle.position)

            # Global best swarm validation
            if particle.score < global_best_score:
                global_best_score = particle.score
                global_best_position = np.copy(particle.position)

        # Vectorized velocity adjustment operations
```

```
        for particle in swarm:
            r1 = np.random.rand(dimensions)
            r2 = np.random.rand(dimensions)

            cognitive_acceleration = c1 * r1 * (particle.best_position - particle.position)
            social_acceleration = c2 * r2 * (global_best_position - particle.position)

            particle.velocity = (w * particle.velocity) + cognitive_acceleration + social_acceleration
            particle.update_position(min_bound, max_bound)

    return global_best_position, global_best_score


# ========================================
# 4. INTERACTIVE RUN ENGINE
# ========================================
if __name__ == "__main__":
    print("=" * 50)
    print("     PSO INVENTORY REPLENISHMENT CONFIGURATOR     ")
    print("=" * 50)

    try:
        # Business Logistics Configurations
        print("\n[STEP 1: Logistical Costs]")
        holding_input = float(input(" -> Holding cost per unit per period: "))
        setup_input = float(input(" -> Fixed ordering/setup cost per batch: "))
        penalty_input = float(input(" -> Stockout penalty cost per shorted unit: "))
        capacity_input = float(input(" -> Maximum procurement capacity / ceiling limit: "))

        demand_str = input(" -> Demand Profile over time (separated by commas, e.g., 20,40,15,80): ")
        demand_forecast = [float(x.strip()) for x in demand_str.split(",")]

        # Hyperparameter Configurations
        print("\n[STEP 2: Swarm Engine Architecture]")
        n_particles = int(input(" -> Number of particles in the swarm (e.g., 30-50): "))
        iterations = int(input(" -> Total computation iterations (e.g., 100-500): "))
        inertia_w = float(input(" -> Inertia weight constant 'w' (e.g., 0.7): "))
        cognitive_c1 = float(input(" -> Personal cognitive parameter 'c1' (e.g., 1.5): "))
        social_c2 = float(input(" -> Social community parameter 'c2' (e.g., 1.5): "))

        print("\nOptimizing logistical supply paths...")

        # Execute custom PSO run
        best_schedule, min_total_cost = pso_stock_optimization(
            demand_forecast=demand_forecast
```

```
            holding_cost=holding_input,
            setup_cost=setup_input,
            penalty_cost=penalty_input,
            max_capacity=capacity_input,
            num_particles=n_particles,
            max_iterations=iterations,
            w=inertia_w,
            c1=cognitive_c1,
            c2=social_c2
        )

        # Render clean results panel
        print("\n" + "=" * 50)
        print("          OPTIMIZED PROCUREMENTS RESULT           ")
        print("=" * 50)
        for period, qty in enumerate(best_schedule, 1):
            order_qty = round(qty) if qty > 0.5 else 0
            print(f" Period {period:02d}: Procure {order_qty:<5} units (Expected Demand: {demand_forecast[period-1]})")

        print("-" * 50)
        print(f" Optimized Minimum System Cost: ${min_total_cost:,.2f}")
        print("=" * 50)

    except ValueError as err:
        print(f"\n[Configuration Error]: Please verify input metrics are strictly numeric values. ({err})")
```

```
==================================================
     PSO INVENTORY REPLENISHMENT CONFIGURATOR
==================================================

[STEP 1: Logistical Costs]
 -> Holding cost per unit per period: 4
 -> Fixed ordering/setup cost per batch: 25
 -> Stockout penalty cost per shorted unit: 5
 -> Maximum procurement capacity / ceiling limit: 450
 -> Demand Profile over time (separated by commas, e.g., 20,40,15,80): 34,67,90,123

[STEP 2: Swarm Engine Architecture]
 -> Number of particles in the swarm (e.g., 30-50): 47
 -> Total computation iterations (e.g., 100-500): 6
 -> Inertia weight constant 'w' (e.g., 0.7): 0.2
 -> Personal cognitive parameter 'c1' (e.g., 1.5): 2.0
 -> Social community parameter 'c2' (e.g., 1.5): 1.0
```

```
Optimizing logistical supply paths...

==================================================
          OPTIMIZED PROCUREMENTS RESULT
==================================================
 Period 01: Procure 35    units (Expected Demand: 34.0)
 Period 02: Procure 65    units (Expected Demand: 67.0)
 Period 03: Procure 89    units (Expected Demand: 90.0)
 Period 04: Procure 204   units (Expected Demand: 123.0)
--------------------------------------------------
 Optimized Minimum System Cost: $441.53
==================================================
```